

GC-free. ODINANN supports *GC-free inserts*. Compared to the log-structured layout, ODINANN directly reuses the invalid records, eliminating the need for *logical* garbage collection – a process to gather valid records into a new index. Our designs target FTL-based SSDs and do not interfere with their *physical* garbage collection. For *deletes*, to recycle the space of deleted vectors, ODINANN needs the lightweight merge (§3.4.2), which is similar to a logical GC.

Insert latency. Compared to buffered insert in DiskANN, ODINANN has a higher insert latency as it directly writes the updated records to disk. This leads to a ~11.1ms insert latency (§4.4). We believe this latency is acceptable, as real-world systems [7, 22] target indexing latencies around 10ms.

Memory usage. ODINANN stores three things in memory:

- *PQ table* for PQ-compressed vectors, 32 bytes per vector.
- *ID-to-location and ID-to-tag hash tables* contain the mapping of vector ID to 4-byte record location and 4-byte tag (the same as DiskANN [23]), 16 bytes per vector.
- *Location-to-ID hash table* contains the mapping of record location to vector ID, used by the first pass of a merge. We use per-page fixed-size arrays for this, requiring 4 bytes per record (including empty ones) and 4 bytes per page.

For a typical ODINANN index with 2× disk space consumption and 6 records per page, the memory usage is ~58 bytes per vector or 58GB for billion-scale datasets. The memory usage in our evaluation is higher (83.8GB, §4.3). This is due to the memory overheads of hash tables, and free slots caused by dynamic doubling of the PQ table and hash tables.

4 Evaluation

We evaluate ODINANN to answer the following questions:

- How does ODINANN compare to other updatable ANNS indexes for concurrent inserts and searches? (§4.2)
- How does ODINANN perform in billion-scale datasets compared to other ANNS indexes? (§4.3)
- How do the techniques in ODINANN contribute to the performance of direct insert? (§4.4)
- How do ODINANN’s techniques for inserts impact internal metrics, e.g., disk consumption and index quality? (§4.5)
- How does the number of out-neighbors impact ODINANN’s insert and search performance? (§4.6)
- How does ODINANN perform for workloads with concurrent inserts, deletes, and searches? (§4.7)

4.1 Experimental Setup

Basic configuration. We use one node for evaluation, which has the following configuration:

- **CPU:** 2× 28-core Intel Xeon Gold 6330 @ 2.00GHz;
- **RAM:** 512GB (16× 32GB DDR4 2933MT/s);

- **SSD:** 1× Samsung PM9A3 3.84TB;
- **OS:** Ubuntu 22.04 LTS with Linux kernel 5.15.0.

Datasets. We use three vector datasets for evaluation:

- SIFT100M [12], a dataset containing 100 million `uint8` vectors with a dimension of 128, and 10,000 query vectors.
- DEEP100M [3], a dataset containing 100 million `float` vectors with a dimension of 96, and 10,000 query vectors.
- SIFT1B [12], a dataset containing one billion `uint8` vectors with a dimension of 128, and 10,000 query vectors.

SIFT100M and DEEP100M are subsets of SIFT1B and DEEP1B with 100 million vectors.

Compared systems. We compare ODINANN with two state-of-the-art ANNS indexes: DiskANN [23] and SPFresh [30].

DiskANN is a graph-based index using buffered inserts. ODINANN is implemented based on DiskANN. Unless otherwise stated, we use the following update configurations for them: Insert candidate pool size l_i is 128. Maximum out-neighbors per record R is 96 for SIFT100M and DEEP100M, and 128 for SIFT1B. The maximal number of vectors to prune in delete C is 384. DiskANN merges updates when the in-memory index size reaches 6% of the on-disk index.

For search configurations, they use a beamwidth equal to 4. The default search candidate list l_s is set to 20 for SIFT100M, and 25 for DEEP100M and SIFT1B. The near-root neighbor cache is disabled. For efficiency, we use `io_uring` [13] in replace of `libaio` as their I/O engine.

SPFresh is a variant of SPANN [5], a cluster-based index that divides vectors into clusters. It supports vector updates by updating the target vector in its nearest clusters. To reduce the imbalance of clusters, SPFresh splits and merges them. We configure SPFresh using the same settings as its open-source code. The maximum cluster size is set to 16KB for SIFT and 48KB for DEEP, for the same number of vectors in a cluster.

We do not compare with other on-disk ANNS systems, such as Starling [27], as they are not designed to support updates.

Metrics. ODINANN is designed for online ANNS scenarios, so we evaluate the search performance, search accuracy, and memory usage during concurrent updates. We use multiple search threads to evaluate search performance, as high search throughput is required in real-world workloads (e.g., 5000 QPS [29]). We gradually add search threads to increase throughput while keeping latency relatively low. For ODINANN and DiskANN, we use 32 search threads. For SPFresh, we use 16 search threads in SIFT and 8 threads in DEEP.

4.2 Overall Performance: Insert-Search

In this evaluation, we build indexes using SIFT100M and DEEP100M datasets and then insert another 100 million vectors into them. Multi-thread searches are concurrently executed to show their interference by inserts and merges.

The evaluated systems take different times in the evaluations. This is because we do not insert vectors during

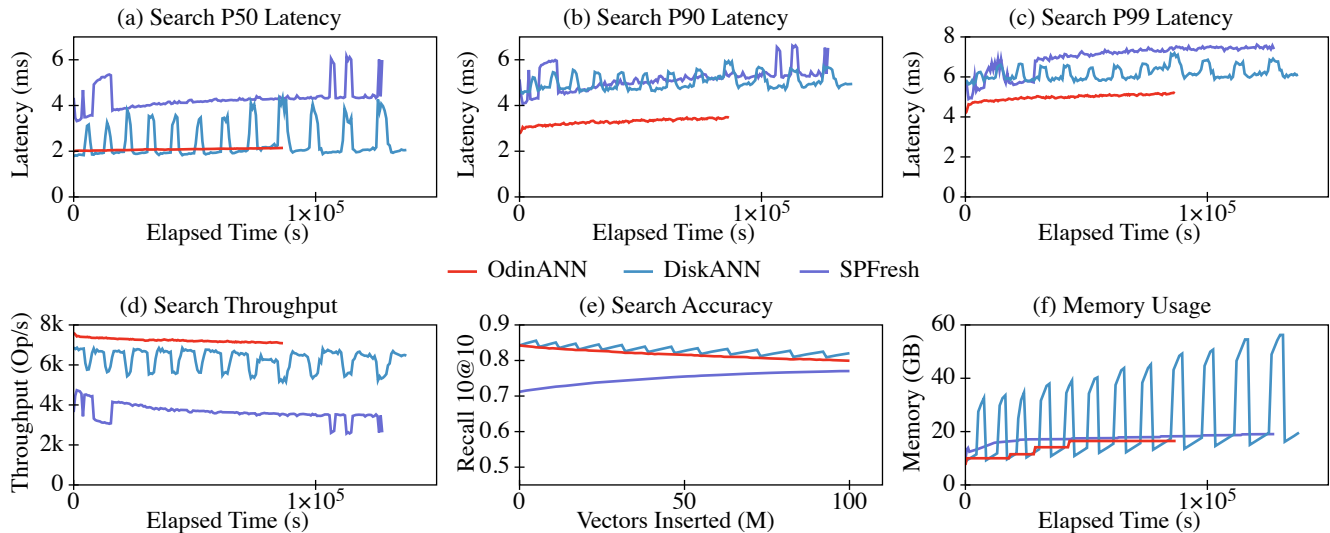


Figure 6: Overall performance in SIFT100M dataset, insert another 100M vectors.

DiskANN’s merge, and the average insert throughputs of compared indexes are different. We set their insert throughputs to 800~1200 QPS, similar to previous works [30].

4.2.1 SIFT100M

Figure 6 shows the results in the SIFT100M dataset. From the figure, we make the following observations:

Low and stable search latency. The results are shown in Figure 6(a)~Figure 6(c). Compared with DiskANN, ODINANN shows lower P50 (median) search latency with less fluctuation (i.e., maximum P50 latency / minimum P50 latency). ODINANN has 13.3% lower P50 latency than DiskANN on average, which shows that direct insert introduces little cost for disk writes compared to merging them. Besides, the P50 latency of ODINANN fluctuates only 1.07 $\times$ at most, compared to 2.44 $\times$ in DiskANN. This is because merging in DiskANN causes bandwidth interference with search tasks, which increases the search latency.

Compared with DiskANN, ODINANN has 34.6%/19.5% lower P90/P99 search latency on average, more significant than the P50 latency. This is because of read-write conflicts of DiskANN’s in-memory index, which uses per-node read-write locks for concurrency control. An insert acquires the per-node lock and blocks concurrent readers. Contrastively, ODINANN uses approximate concurrency control to reduce such blocking. Writers only block readers when they are updating the id-to-location mapping.

Compared with SPFresh, ODINANN has 51.7%/36.5%/28.4% lower P50/P90/P99 search latency on average. This is because fine-grained graph-based indexes (ODINANN) are advantageous in search compared to coarse-grained cluster-based indexes (SPFresh), especially in disk bandwidth con-

sumption. On average, ODINANN reads 41 pages per search, while SPFresh reads 198 pages. Although SPFresh replicates the clusters to make disk reads mostly sequential, it still shows higher latency than ODINANN.

High search throughput. Compared with DiskANN and SPFresh, ODINANN has 1.15 $\times$ and 1.99 $\times$ throughput on average, as in Figure 6(d). The reason here is the same as that in latency. DiskANN needs extra search in the in-memory index, and it suffers from bandwidth interference during merge. The cluster-based SPFresh consumes more disk IOPS than ODINANN per search, which limits its throughput.

Unsacrificed search accuracy. As shown in Figure 6(e), the accuracy of ODINANN is comparable with DiskANN. The accuracy of ODINANN is 99.1%~100% that of DiskANN when the in-memory index is empty. The accuracy gap between ODINANN and DiskANN increases along with the in-memory index size, as DiskANN finds top- k in both the in-memory and on-disk indexes for each search. However, the in-memory index forces DiskANN to merge updates to disks, which causes performance fluctuation.

ODINANN and DiskANN consistently show higher accuracy than SPFresh. This is because graph-based indexes are more advantageous than cluster-based indexes in terms of search [7]. We show that this advantage still exists even after the index is updated. In all, ODINANN maintains lower latency, higher throughput, and higher search accuracy than SPFresh.

The accuracy of ODINANN and DiskANN drops with time. This is because they use PQ to accelerate pruning in insert operations, which is less accurate than the initial graph building [23]. Another reason is that larger datasets intrinsically require a larger search L for higher accuracy.

The accuracy of SPFresh increases over time, because of its

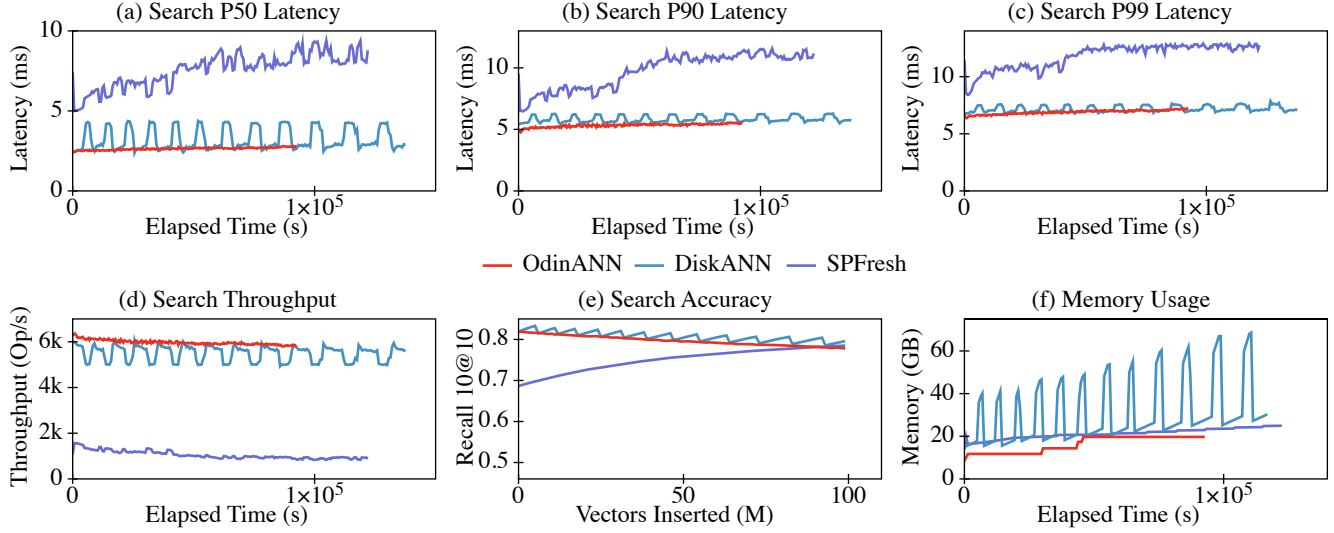


Figure 7: Overall performance in DEEP100M dataset, insert another 100M vectors.

cluster split protocol. However, it comes at the cost of more pages accessed per request, which results in a throughput drop over time, as shown in Figure 6(d).

Low memory footprint. ODINANN’s peak memory usage is 29.3% of DiskANN. This is because ODINANN uses direct inserts. ODINANN only needs an extra ID-to-location mapping in memory to support out-of-place record updates, while DiskANN stores the in-memory index and the delta of updates for merging. They lead to a 56.3GB peak memory usage in the final index with ~ 200 M vectors. The in-memory data structures in DiskANN require even more memory when the on-disk index size further increases, to keep a fixed merge threshold (6% of the on-disk index size in our configuration).

Compared with SPFresh, ODINANN has 86.8% peak memory usage. SPFresh stores a navigation graph for indexing the nearby clusters of the target vector. The graph size increases when a cluster split happens.

4.2.2 DEEP100M

Figure 7 shows the result in the DEEP100M dataset. In this dataset, ODINANN keeps low and stable latency, and low memory usage. We make further observations:

(1) The performance boost of ODINANN in DEEP100M is less significant than in SIFT100M. This is because it benefits less from out-of-place record updates. Each vector in DEEP100M is 384B, compared to 128B in SIFT100M. Larger vectors result in fewer records on a page (5 vs. 7), and thus fewer merge opportunities for updates.

(2) The advantage of ODINANN over SPFresh is more significant in DEEP100M, due to larger vectors. ODINANN and DiskANN only use one vector per search step, whose size is smaller than a page. Therefore, large vectors have little

impact on performance. Their performance drop mainly accounts for the change of search L . However, SPFresh executes a brute-force search for all the vectors in a cluster. In this configuration, we use the same number of per-cluster vectors as SIFT. Larger vectors result in more per-cluster disk I/O, which is the bottleneck of SPFresh’s search performance.

4.3 Overall Performance: SIFT1B

In this section, we show that ODINANN efficiently handles billion-scale datasets. We build an initial index with 800M vectors in SIFT1B. Then, we insert the other 200M vectors into it and execute concurrent searches. DiskANN merges the in-memory index to disk when it reaches 30 million vectors ($\sim 3\%$), the same as its paper [23].

Figure 8 shows the results. From the figure, we find that the stable performance of ODINANN holds, so we can draw similar conclusions as in §4.2. For example, ODINANN shows 85.7% and 62.1% median latency on average compared to DiskANN and SPFresh. We make two new observations:

First, for DiskANN, although the number of vectors per merge is reduced from 6% (in §4.2) to 3% of the on-disk index, it still shows a high memory usage (> 200 GB). Also, P99 search latency spikes are observed, due to bandwidth contention when merging to disk. Second, SPFresh uses less memory than ODINANN (52.2GB vs. 83.8GB). This is because SPFresh uses a coarse-grained cross-cluster index. To achieve a higher search accuracy, SPFresh should make its clusters fine-grained, inducing higher memory usage.

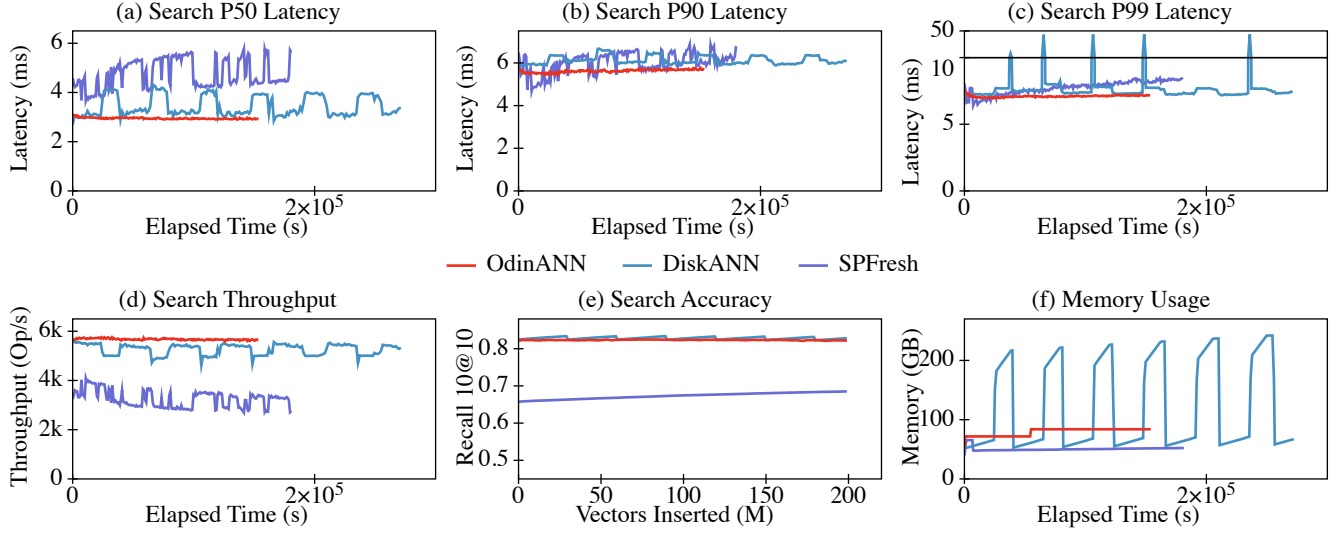


Figure 8: Overall performance in SIFT1B dataset, insert 200M vectors into an index of 800M vectors.

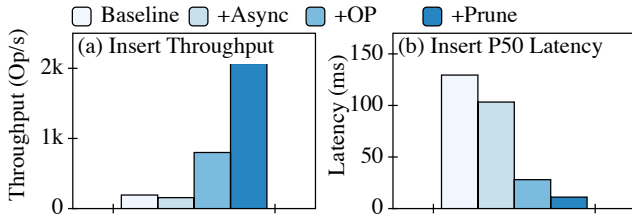


Figure 9: Breakdown analysis of insert.

4.4 Breakdown Analysis

In this section, we break down the performance gap between Baseline and ODINANN, to show the contributions of the techniques to efficient direct insert. We build a base index using the SIFT100M dataset and insert 100K vectors into it. We evaluate insert latency and throughput using 27 insert threads, without concurrent search.

Baseline. We implement the Baseline based on ODINANN. It does direct inserts by updating records in place and uses the basic approximate concurrency control, without the two optimizations. We do not use two-phase locking for Baseline, as it seriously degrades the performance. Figure 9 shows that Baseline has low insert throughput, which can not meet the demands of real-world workloads (e.g., 1000 QPS [29]).

+Async. We move the disk I/O out of the critical section (Optimization #1 in §3.3). It reduces the insert latency to 80.8%. However, the insert throughput can not be increased because of the inherent cost of inserts.

+OP. We do space overprovision and out-of-place record updates (§3.2). It increases the insert throughput to 5.12 $\times$ and reduces its latency to 27.1%. This is because it not only reduces the inherent cost of disk writes by combining them, but reduces contention by updating the pages with fewer records.

+Prune. We use delta pruning (Optimization #2 in §3.3) to reduce the compute overhead. It further increases the throughput to 2.59 $\times$ and reduces the latency to 39.7%. By adopting all the optimizations, ODINANN reaches 2000 QPS with 11.1ms median latency for insert, which meets the demands of real-world workloads better.

4.5 In-Depth Analysis

To further understand the tradeoffs in ODINANN, we collect statistics for the indexes in §4.2 on disk space consumption and index quality.

Disk space consumption. ODINANN does space overprovision for fewer writes. Here, we compare the disk space consumption of ODINANN and SPFresh. We exclude DiskANN as its on-disk index is immutable and thus with ideal size.

Figure 10(a) shows the disk space consumption of ODINANN and SPFresh for the final index of §4.2 with 200M vectors. (1) ODINANN has 1.98 $\times$ /2.29 $\times$ space amplification for SIFT/DEEP datasets, which concludes that the record allocation policy in ODINANN has near-ideal space amplification (§3.2.3). (2) ODINANN requires 84.8%/49.7% of SPFresh’s disk space for the final index. This is because SPFresh uses vector replication (8 $\times$ by default) for better search accuracy, which consumes more space than the edges in ODINANN. Also, SPFresh splits clusters into half-empty ones for cluster balancing, which causes space amplification.

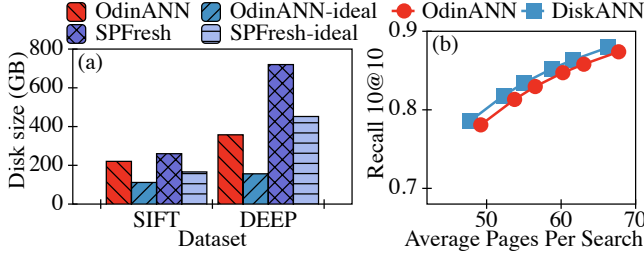


Figure 10: (a) Disk space consumption of ODINANN and SPFresh compared to the ideal case (no overprovision and split). (b) Index quality of ODINANN and DiskANN after inserting 93 million vectors into an index of DEEP100M.

Index quality. Approximate concurrency control uses a relaxed isolation level and delta pruning for better concurrency, at the cost of index quality. Figure 10(b) shows the index quality of DEEP100M after the last merge in §4.2.2 (i.e., 93M vectors inserted). With the same recall, ODINANN accesses ~4.5% more disk pages than DiskANN. The performance loss is acceptable considering the benefits of direct inserts, and it can be further optimized by a more accurate algorithm for delta pruning, which we leave for future work.

4.6 Parameter Study: Out-Neighbors

In this section, we show how the number of maximum out-neighbors (R) impacts insert and search performance. We build indexes on SIFT100M using different R s (32, 64, 96, 128), where the maximum R is used by previous works [24] and ODINANN to support billion-scale datasets. The insert candidate pool size l_i is set to $R + 32$ (e.g., 128 for $R = 96$). We use 10 insert threads and 32 search threads for concurrent insert and search, the same as in §4.2.1.

Figure 11 shows the results. From the figure, we observe that: (1) A large R increases search performance but reduces insert throughput. A large R contributes to better graph connectivity, which leads to reduced search latency. However, it requires a large insert candidate pool (l_i) to find enough neighbors. Thus, an insert requires to search more vectors, decreasing its performance. As shown in the figure, $R = 96$ strikes a balance between search performance and insert throughput, so we choose $R = 96$ as our SIFT100M configuration.

(2) The memory usage is nearly constant across different R s, as discussed in §3.5. However, setting R to 128 results in ~800MB (8.75%) more memory usage compared to $R = 64$. This is due to our implementation of the location-to-ID table, which uses per-page arrays with size 10. A larger R leads to more pages in the index, and thus a larger location-to-ID table. When $R = 32$, we use a larger array size of 15. It shows higher memory usage than $R = 64$, due to the dynamic doubling of tables during insert. We consider the difference

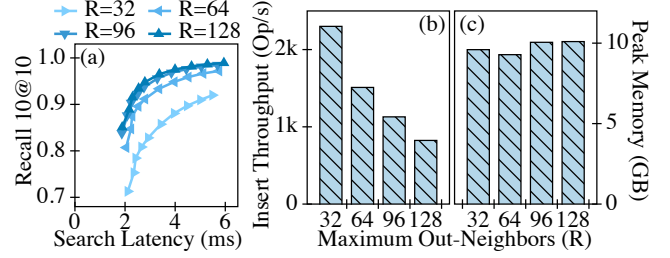


Figure 11: The impact of maximum out-neighbors (R) on (a) search latency; (b) insert throughput; (c) memory usage. 10 insert threads and 32 search threads.

in memory usage is not a major issue, as we can always select an appropriate per-page array size for each graph index.

4.7 Workload with Deletes

In this section, we show the performance of ODINANN in workload with deletes. We build an index on SIFT100M, and then gradually replace the original 100M vectors with another 100M vectors. ODINANN merges at 20% updated vectors, while DiskANN merges at 6%. We do not update vectors during merge, to directly show the interference of merge on the frontend search. We show the results for similar durations.

The result is shown in Figure 12, in which the performance of SPFresh is similar to that in §4.2. We compare ODINANN with DiskANN and make the following observations:

(1) The merge period of ODINANN has little interference on frontend search, as merging deletes only requires two scans of the on-disk index, which is not I/O-intensive. However, merging inserts in DiskANN is I/O intensive, so it has bandwidth interference with frontend search tasks.

(2) Merging in ODINANN can be less frequent than in DiskANN because buffered deletes require much less memory than buffered inserts during runtime. Deleting a vector only records an ID in memory, while inserting records the vector and its edges in the in-memory index.

(3) ODINANN uses less memory than DiskANN during a merge. This is because ODINANN only loads 20 nearest neighbors for each deleted node into memory, while DiskANN loads all the 96 neighbors. The neighbor number can be configured to trade index quality (~2.5% accuracy for the final index, with the same number of disk I/Os) off for memory. An accurate and memory-efficient delete method is preferred to optimize this, which we leave for future work.

(4) The search accuracy of ODINANN can be ensured between two merges, thanks to the dynamic candidate pool design. With a fixed configuration of candidate pool length, ODINANN can ensure the search accuracy regardless of deleted vectors, at the cost of more disk I/O per search.

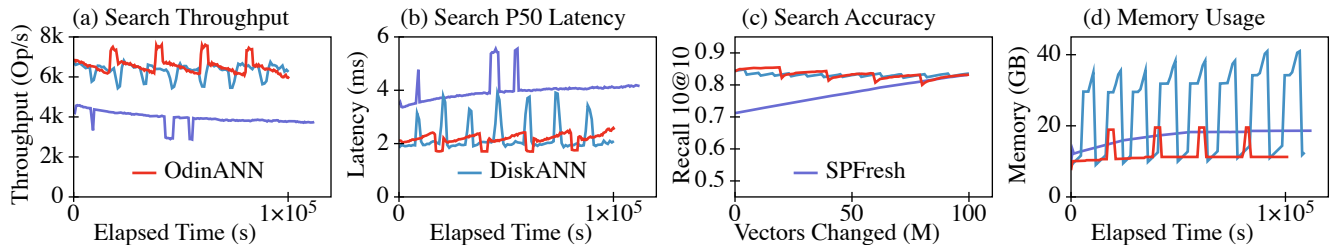


Figure 12: Workload with deletes in SIFT100M dataset, replace the original 100M vectors with another 100M vectors. Note that ODINANN merges at 20% updated vectors, while DiskANN merges at 6% for less memory usage. With similar elapsed time in this figure, ODINANN and SPFresh finish replacing the whole 100M vectors, while DiskANN replaces 50M vectors.

5 Related Work

Billion-Scale ANNS indexes. Real-world workloads do vector search on billions of vectors [7, 22]. Traditional ANNS indexes are stored in memory [4, 7, 18]. To support billion-scale datasets in a single node, they suffer from high prices and limited scalability. To this end, two new methods, *hybrid storage* and *vector compression*, are proposed.

Hybrid storage. Some works rely on hybrid storage media for more capacity, such as SSD and DRAM. DiskANN [24] and Starling [27] are on-disk graph-based indexes. They store the full graph on disk, and compressed vectors in memory for graph navigation. SPANN [5] is an on-disk cluster-based index. It stores the clusters on disk, and a small graph in memory to index the nearest clusters. HM-ANN [21] maintains a hierarchical graph [18] in DRAM and persistent memory.

Some other works seek performance boosts with near-data processing. CXL-ANNS [11] uses processing units near the CXL memory for distance calculation, and SmartANNS [25] uses FPGAs in SmartSSDs for graph traversal. These methods can not only reduce the CPU burden for computation, but mitigate the bandwidth bottleneck by avoiding transferring vectors between CPU and slow-speed storage media.

ODINANN uses commodity SSDs for capacity expansion. It does not rely on near-data processing for better compatibility.

Vector compression. Instead of increasing capacity, some works compress vectors for smaller index sizes in memory. Product quantization (PQ) [8, 14] is a popular vector compression method for ANNS. It splits vectors into equally sized chunks and does clustering for each chunk. Vectors are replaced with per-chunk IDs representing their nearest centroids. This way is used by IVF [2] indexes for less memory footprint. DiskANN [24] also stores PQ-compressed vectors in memory to reduce neighbor reads during graph navigation. LVQ [1] focuses on single-vector compression instead of cross-vector clustering. It uses scalar quantization in each vector to map floating-point vectors to bits, which improves search performance with reduced memory footprint.

ODINANN stores PQ-compressed vectors in memory for

graph navigation. The designs in ODINANN are orthogonal to vector compression methods.

Updatable ANNS indexes. Graph-based DiskANN [23] uses buffered inserts, which interfere with frontend search tasks, motivating our designs for ODINANN. Cluster-based SPFresh [30] supports in-place vector updates. It is more challenging for graph-based ODINANN to support efficient updates than cluster-based SPFresh, due to the intrinsic complexity of graph neighbor maintenance. However, ODINANN benefits from better search accuracy and performance.

Reserving space for efficient updates. ALEX [6] is a learned index that also leverages reserved space (called "gaps" in its paper) for efficient updates. Unlike ALEX, which uses one gap for each new key-value pair to avoid model retraining, ODINANN uses multiple record slots for a single insert, combining record updates in the same page.

6 Conclusion

We argue that direct insert is more suitable for graph-based ANNS indexes and propose ODINANN. It efficiently processes inserts in the on-disk index instead of buffering them. Evaluations show that ODINANN keeps stable insert and search performance in billion-scale vector datasets. This work demonstrates that on-disk graph-based ANNS indexes can benefit from high search accuracy, stable search performance, as well as efficient updates simultaneously using direct insert.

Acknowledgments

We sincerely thank our shepherd, Bingzhe Li, and the anonymous reviewers for their valuable feedback. This work is supported by the National Key R&D Program of China (Grant No. 2024YFE0203300), the National Natural Science Foundation of China (Grant No. 62332011), and Beijing Natural Science Foundation (Grant No. L242016).